

1. Question

Write a Java program to demonstrate exception handling using `ClassCastException` when an object is incorrectly cast to another data type.

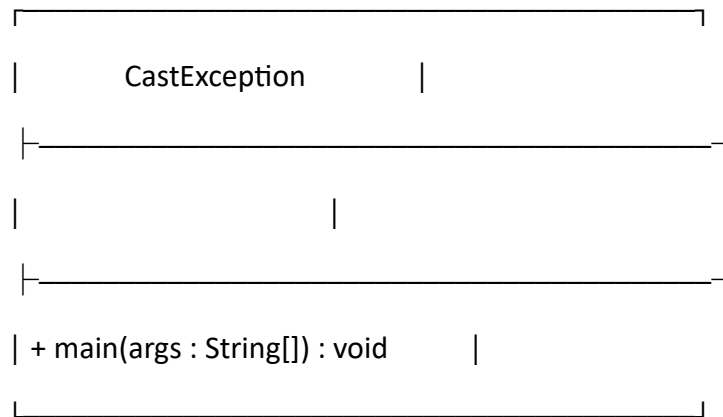
2. Steps for Execution

1. Open a Java IDE or command prompt.
 2. Create a package named `javapractice`.
 3. Create a class named `CastException`.
 4. Create an Object containing the string "Hello".
 5. Use a try block to convert the object into an Integer.
 6. Since the actual object is a String, it cannot be cast to Integer.
 7. Java generates a `ClassCastException`.
 8. Handle the exception using the catch block.
 9. Display the appropriate error message.
 10. Compile and execute the program.
-

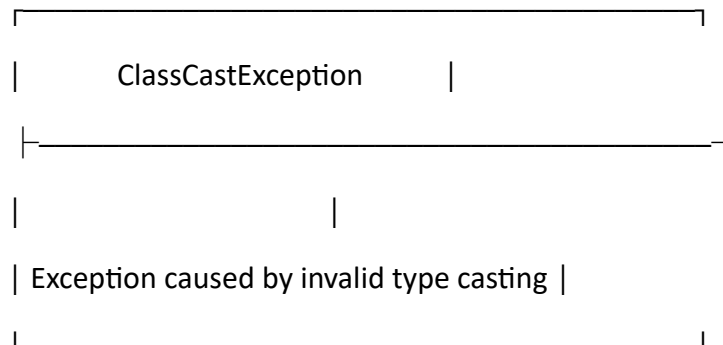
3. Algorithm

1. **Start**
 2. Create an Object variable and store "Hello" in it.
 3. Begin the try block.
 4. Display "Converting object into Integer".
 5. Try to cast the Object into Integer.
 6. Since the actual object is a String, a `ClassCastException` occurs.
 7. Catch the exception using the catch block.
 8. Display "Invalid type casting".
 9. **Stop**
-

4. UML Class Diagram



catches



5. Code

```
package javapractice;
```

```
public class CastException {
```

```
    public static void main(String[] args) {
```

```
        Object obj = "Hello";
```

```
        try {
```

```
            System.out.println("Converting object into Integer");
```

```
            Integer num = (Integer) obj;
```

```
            System.out.println(num);
```

```
    }  
    catch (ClassCastException e) {  
        System.out.println("Invalid type casting");  
    }  
}  
}
```

6. Output

Converting object into Integer

Invalid type casting

7. Result

Thus, the Java program successfully demonstrates exception handling using ClassCastException when an object is incorrectly cast from String to Integer.